Dimensionality Reduction and Clustering

1. Executive Summary

This report examines the use of two dimensionality reduction techniques—Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE)—alongside two clustering methods, K-means and DBSCAN, to enhance visualization and classification of the Iris dataset's three species. The objective is to explore how PCA and t-SNE simplify data visualization by reducing dimensions and to assess how K-means and DBSCAN classify species within this reduced space. By comparing the strengths and limitations of each technique, the report aims to guide method selection for analyzing complex datasets with diverse structures and clustering needs.

2. Preprocessing the Data

a) Data Import and Labeling: Load the Iris dataset and assign clear column names for better readability.

```
import pandas as pd
data = pd.read_csv('iris/iris.data', header=None)
# add column names
data.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
data
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

The Iris dataset contains 150 samples of iris flowers, each described by four features: *sepal_length, sepal_width, petal_length*, and *petal_width*, all measured in centimeters. These features provide measurements of the physical dimensions of the flowers. It is divided into three classes (or species) of iris flowers: *Iris-setosa, Iris-versicolor, and Iris-virginica*, with 50 samples in each class.

b) Feature and Label Separation: Separate the feature columns (*sepal_length, sepal_width, petal_length, petal_width*) from the target label (*species*), preparing the data for PCA.

c) **Standardization**: Normalize the feature values using *StandardScaler* to ensure each feature has a mean of 0 and a standard deviation of 1. This step prevents any feature from disproportionately influencing the PCA results due to scale differences.

```
# preprocess the dataset by normalizing its features
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
```

3. Principal Component Analysis (PCA)

3.1 PCA Implementation and Visualization

PCA was applied to the standardized data. In this process, we calculated the principal components, which are linear combinations of the original features. Each component captures a unique dimension, with the first component explaining the most variance, followed by the second, and so on.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca_results = pca.fit_transform(scaled_features)
```

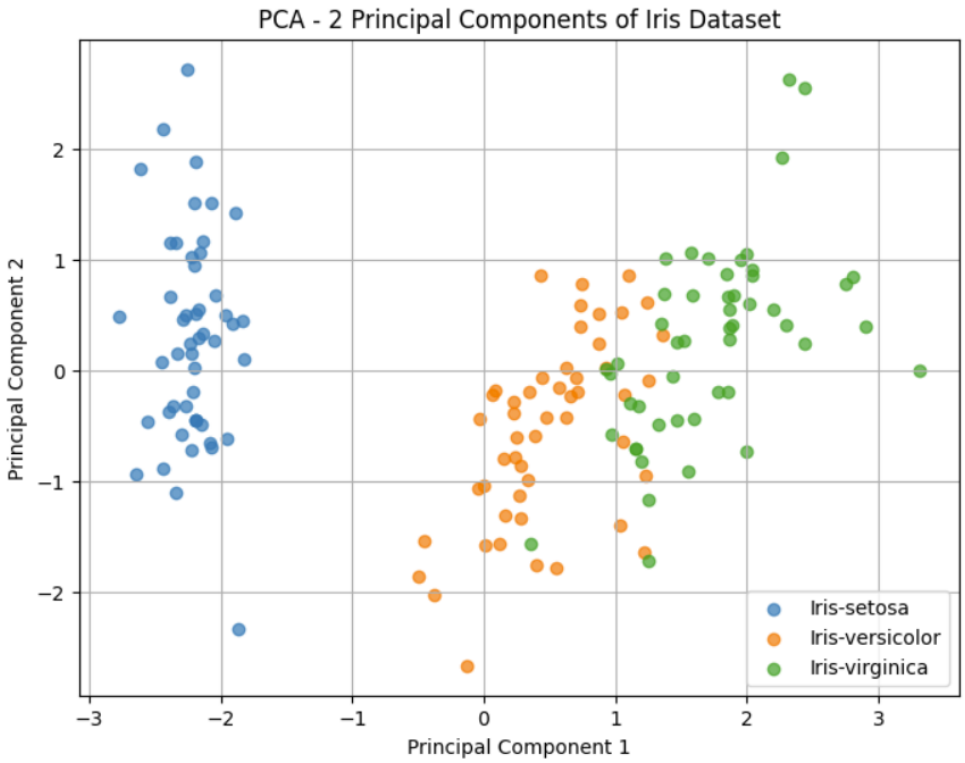
```
import matplotlib.pyplot as plt

# create a DataFrame with the PCA results for visualization
pca_df = pd.DataFrame(data=pca_results, columns=['PC1', 'PC2'])
pca_df['species'] = labels

plt.figure(figsize=(8, 6))
for species in pca_df['species'].unique():
    subset = pca_df[pca_df['species'] == species]
    plt.scatter(subset['PC1'], subset['PC2'], label=species, alpha=0.7)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA - 2 Principal Components of Iris Dataset')
plt.legend()
plt.grid()
plt.show()
```

Visualization:



Analysis:

The PCA plot shows the distribution of the Iris dataset based on the first two principal components. Each color represents one of the three iris species: *Iris-setosa, Iris-versicolor, and Iris-virginica*.

The PCA plot displays the distribution of the Iris dataset based on the first two principal components, with each color representing one of the three species: Iris-setosa, Iris-versicolor, and Iris-virginica. These two components capture most of the dataset's variance, enabling effective dimensionality reduction while preserving the data's essential structure.

The first principal component, shown on the x-axis, captures the largest share of the variance and serves as the primary direction of data variation. This component effectively separates Iris-setosa (blue points) from the other two species, indicating unique characteristics along this axis. The second principal component, on the y-axis, captures additional variance and further differentiates between Iris-versicolor and Iris-virginica. Although it explains less variance than the first component, it contributes to the visualization by separating clusters vertically, helping to identify patterns between these overlapping species.

The distinct clustering of Iris-setosa highlights its unique traits, while the overlap between Iris-versicolor and Iris-virginica suggests similarities that make them harder to separate within this reduced feature space.

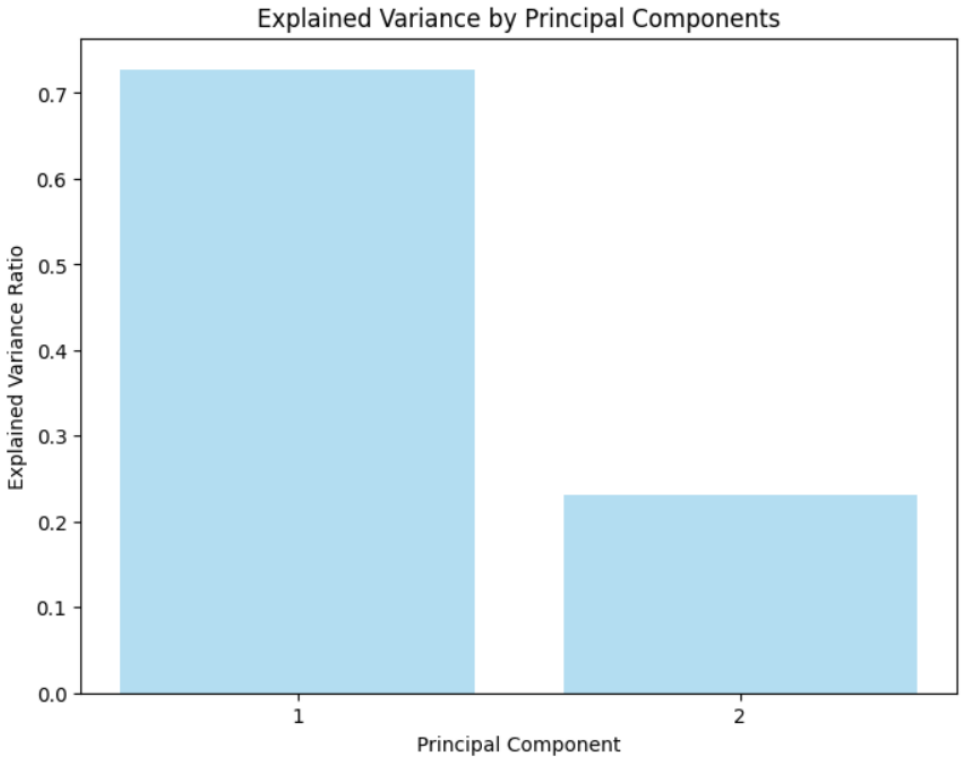
3.2 Variance Explained Ratio

A bar chart displays the variance explained by each component, showing a steep drop after the first two. This supports using only the first two components for further analysis, as they retain most of the dataset's information.

```python
# Calculate the explained variance ratio for each principal component
explained_variance_ratio = pca.explained_variance_ratio_

# Bar chart for explained variance ratio
plt.figure(figsize=(8, 6))
plt.bar(range(1, len(explained_variance_ratio) + 1), explained_variance_ratio, alpha=0.7, color='skyblue')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')
plt.title('Explained Variance by Principal Components')
plt.xticks(range(1, len(explained_variance_ratio) + 1))
plt.show()

# Print the explained variance ratio
for i, variance in enumerate(explained_variance_ratio, start=1):
    print(f"Principal Component {i}: {variance:.2%} of variance explained")
```




```
Principal Component 1: 72.77% of variance explained
Principal Component 2: 23.03% of variance explained
```

Analysis:

The explained variance ratio was calculated for each principal component to show the proportion of total variance they capture. The first component accounted for 72.77% of the variance, and the second for an additional 23.03%, totaling 95.8%. This indicates that nearly all the information in the dataset is represented in just two dimensions.

4. t-SNE

4.1 t-SNE Implementation and Visualization

We applied t-SNE with 2 components to reduce the dataset to a 2D space. Unlike PCA, which maximizes variance, t-SNE preserves local data relationships by minimizing the divergence between high and low dimensional probability distributions.

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, random_state=0)
tsne_results = tsne.fit_transform(scaled_features)
```

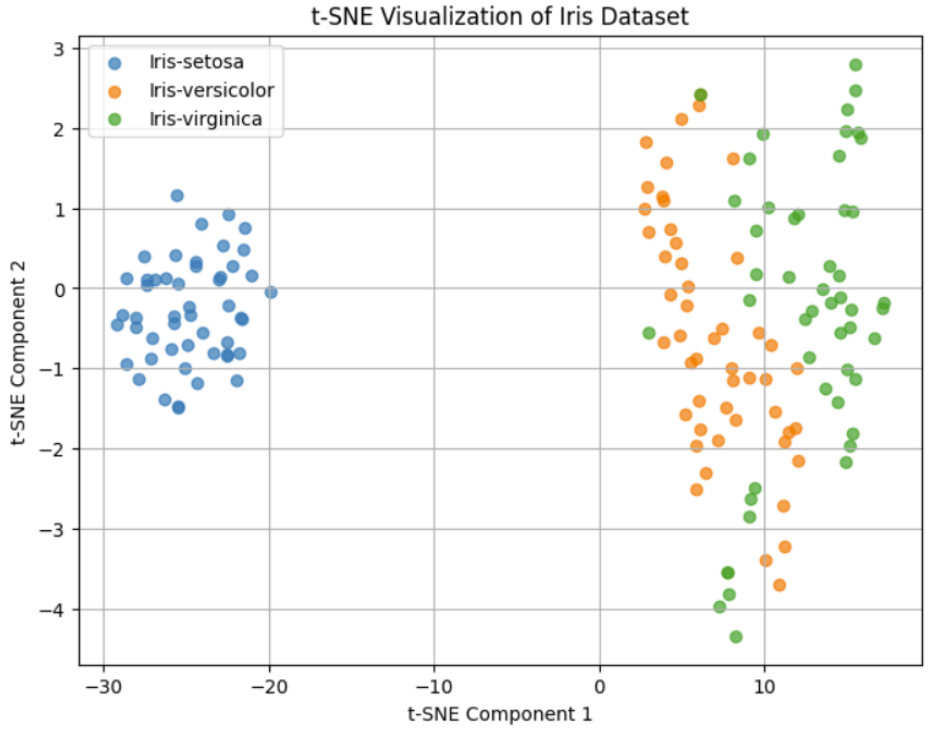
The 2D t-SNE results were plotted, with each species (Iris-setosa, Iris-versicolor, Iris-virginica) represented by a different color, allowing us to examine clustering patterns.

```
import matplotlib.pyplot as plt

# create a DataFrame with the t-SNE results for visualization
tsne_df = pd.DataFrame(data=tsne_results, columns=['TSNE1', 'TSNE2'])
tsne_df['species'] = labels

plt.figure(figsize=(8, 6))
for species in tsne_df['species'].unique():
    subset = tsne_df[tsne_df['species'] == species]
    plt.scatter(subset['TSNE1'], subset['TSNE2'], label=species, alpha=0.7)

plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.title('t-SNE Visualization of Iris Dataset')
plt.legend()
plt.grid()
plt.show()
```



Analysis:

The t-SNE plot provides a 2D visualization of the Iris dataset, effectively grouping each species into distinct clusters.

a) **Distinct Separation of Iris-setosa**: The blue cluster representing Iris-setosa is well-separated from the other species, indicating its unique characteristics in this reduced space.

b) **Overlap Between Iris-versicolor and Iris-virginica**: The orange (Iris-versicolor) and green (Iris-virginica) clusters show some overlap, suggesting that these species share certain features, though t-SNE has managed to separate them to an extent.

c) **Preservation of Local Structure**: The compact clusters highlight t-SNE's strength in preserving local relationships, allowing for a fine-grained view of the species distinctions.

4.2 Comparison with PCA

t-SNE and PCA both reduce the dimensionality of the Iris dataset, but they achieve this in different ways, leading to distinct results.

a) **Methodology:** PCA is a linear technique that maximizes variance along principal components, capturing global structures in the data, but may miss subtle differences within clusters. In contrast, t-SNE is a non-linear technique that focuses on preserving local relationships, making it better suited for visualizing clusters.

b) **Cluster Separation/Visualization:** Both methods distinctly separate *Iris-setosa* from the other species due to its unique features. However, t-SNE produces more compact and well-defined clusters for each species, revealing finer distinctions. In contrast, PCA shows greater overlap between *Iris-versicolor* and *Iris-virginica*, as it focuses on maximizing variance rather than emphasizing local structure.

c) **Interpretability:** PCA provides a straightforward interpretation of variance explained by each component, helping to understand the contribution of each dimension. However, t-SNE is more complex, as it doesn't retain variance in the same way, focusing instead on local patterns without preserving global distances.

d) **Scalability:** PCA is computationally efficient for large datasets because it uses a linear approach based on eigenvalue decomposition. In contrast, t-SNE is more computationally demanding, as it involves calculating pairwise distances and iterative optimization, making it slower and less suitable for large datasets. Therefore, PCA is generally preferred for large-scale analysis, while t-SNE is better suited for smaller datasets where detailed clustering is essential.

5. K-Means Clustering

5.1 K-Means Implementation and Visualization

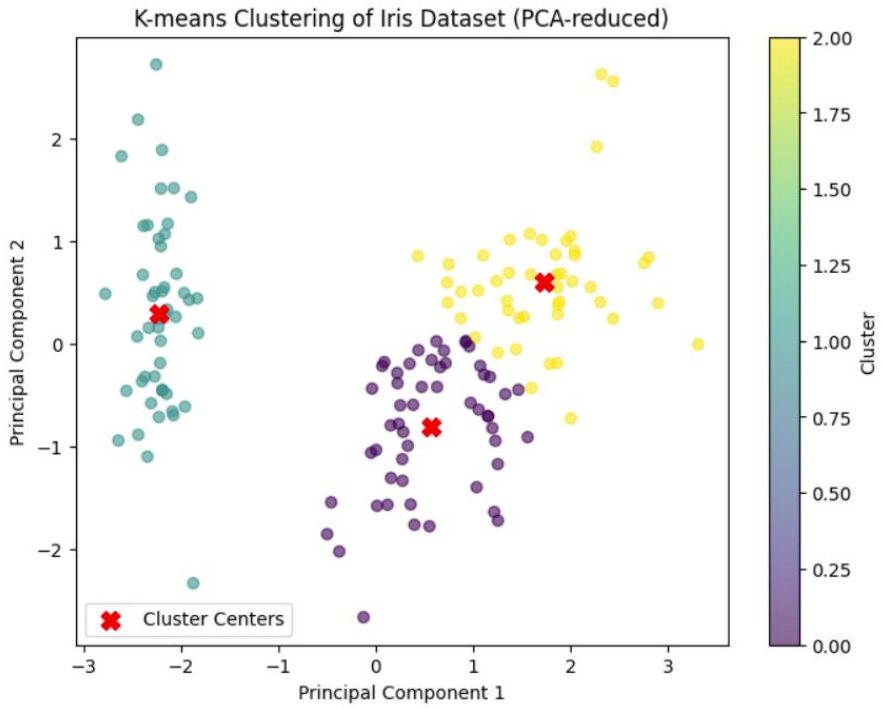
We applied K-means clustering with a predefined number of clusters set to 3, corresponding to the three known species in the dataset. The algorithm iteratively assigns each data point to the nearest cluster center and adjusts the centers to minimize within-cluster variance. Using the PCA-reduced data as input allows us to visualize the clustering results effectively in two dimensions.

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=0)
kmeans_labels = kmeans.fit_predict(pca_results)
```

```
# Add cluster labels to a DataFrame
pca_df['kmeans_cluster'] = kmeans_labels

plt.figure(figsize=(8, 6))
scatter = plt.scatter(pca_df['PC1'], pca_df['PC2'], c=pca_df['kmeans_cluster'], cmap='viridis', alpha=0.6)
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=100, marker='X', label='Cluster Centers')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('K-means Clustering of Iris Dataset (PCA-reduced)')
plt.legend()
plt.colorbar(scatter, label='Cluster')
plt.show()
```



Analysis:

The K-means clustering on the PCA-reduced Iris dataset shows distinct groupings for each species. The leftmost cluster, likely representing Iris-setosa, is well-separated due to its unique features. However, the clusters on the right, corresponding to Iris-versicolor and Iris-virginica, overlap somewhat, reflecting their shared characteristics. This overlap highlights a limitation of K-means in fully separating species with similar features, though overall, the clustering captures the main structure of the dataset effectively.

5.2 Evaluation of Clustering Performance

```
from sklearn.metrics import silhouette_score

inertia = kmeans.inertia_
silhouette_avg = silhouette_score(pca_results, kmeans_labels)

print(f'Inertia: {inertia}')
print(f'Silhouette Score: {silhouette_avg:.2f}')
```

```
Inertia: 116.10924021401532
Silhouette Score: 0.51
```

Analysis:

The K-means clustering results show an inertia of 116.11 and a silhouette score of 0.51.

The inertia value, which measures the sum of squared distances from each point to its nearest cluster center, indicates the compactness of the clusters. A lower inertia typically suggests more cohesive clusters, though its absolute value depends on the data's scale and dimensionality.

The silhouette score of 0.51, which ranges from -1 to 1, reflects moderate clustering quality. This score suggests that while the clusters are somewhat well-separated, there is still overlap, particularly between Iris-versicolor and Iris-virginica. Overall, the score indicates that K-means achieved reasonable clustering, though further refinement may be needed to improve separation between similar species.

A key limitation of K-means in this context is its reliance on Euclidean distance, which assumes clusters are spherical and similar in size. This assumption does not hold for Iris-versicolor and Iris-virginica, which share similar features and do not form perfectly separate, spherical clusters. Additionally, K-means requires specifying the number of clusters in advance. In this case, we set it to three based on prior knowledge, but this would be a challenge in situations without known labels.

6. DBSCAN Clustering

6.1 DBSCAN Implementation and Visualization

DBSCAN relies on two key parameters: epsilon and minimun samples. The eps parameter sets the maximum distance between points for them to be considered part of the same neighborhood, while min_samples defines the minimum number of points needed to form a dense region, or core point. We tested various combinations of epsilon and minimum samples to examine their impact on clustering outcomes in the PCA-reduced data.

```
# try different DBSCAN parameters
eps_values = [0.1, 0.3, 0.5, 0.7]
min_samples_values = [1, 3, 5]
```

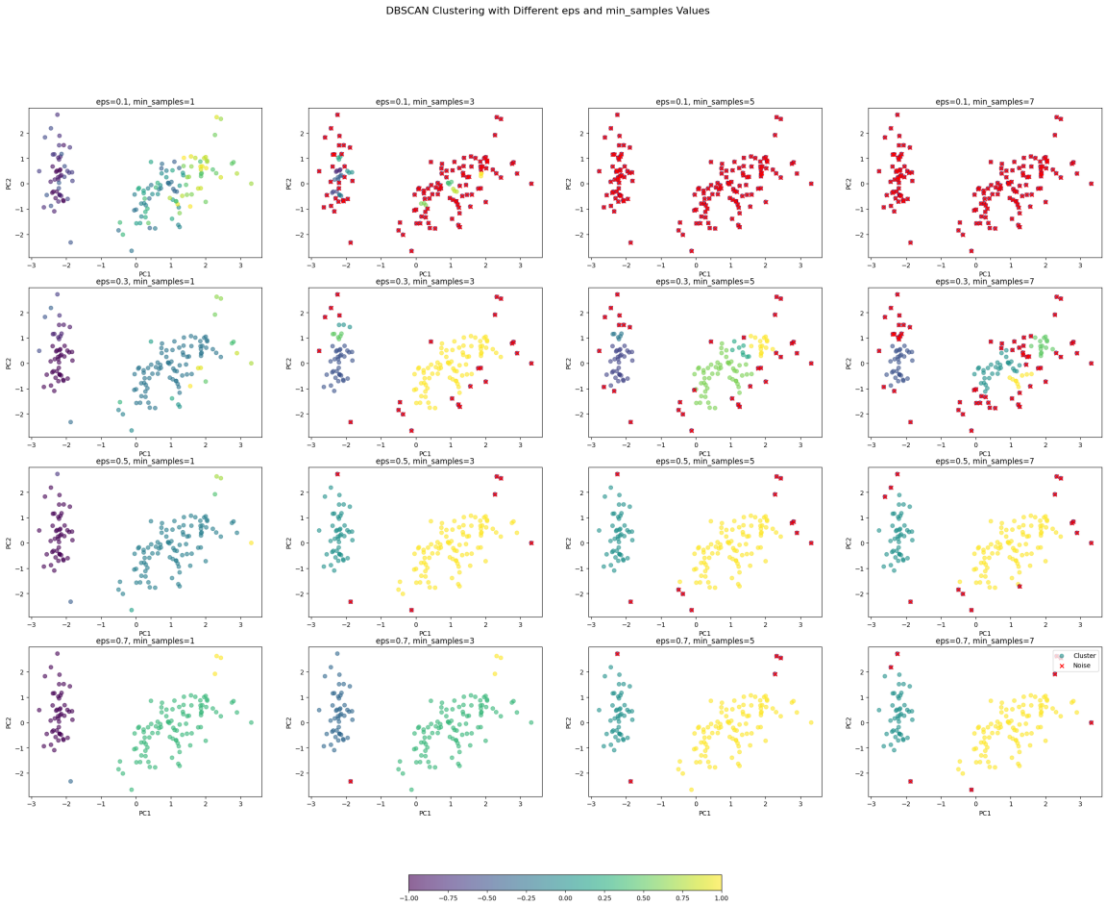
```
# iterate over combinations of eps and min_samples and plot each result
for i, eps in enumerate(eps_values):
    for j, min_samples in enumerate(min_samples_values):
        # Apply DBSCAN with current parameters
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        dbscan_labels = dbscan.fit_predict(pca_results)

        # Convert DBSCAN labels into a DataFrame for visualization
        pca_df = pd.DataFrame(data=pca_results, columns=['PC1', 'PC2'])
        pca_df['dbscan_cluster'] = dbscan_labels
        pca_df['species'] = labels

        # Plot the DBSCAN results in the current subplot
        ax = axes[i, j]
        scatter = ax.scatter(pca_df['PC1'], pca_df['PC2'], c=pca_df['dbscan_cluster'], cmap='viridis', alpha=0.6)
        ax.set_xlabel('PC1')
        ax.set_ylabel('PC2')
        ax.set_title(f'eps={eps}, min_samples={min_samples}')

        # Mark noise points
        noise = pca_df[pca_df['dbscan_cluster'] == -1]
        ax.scatter(noise['PC1'], noise['PC2'], color='red', marker='x', label='Noise')

# add legend and color bar only once for all plots
plt.legend(['Cluster', 'Noise'], loc='upper right')
fig.colorbar(scatter, ax=axes, orientation='horizontal', fraction=0.02, pad=0.1)
plt.show()
```



Analysis:

With smaller eps values, such as 0.1, DBSCAN identifies many points as noise (marked in red) and creates smaller, more fragmented clusters. This indicates that when eps is too low, DBSCAN is overly restrictive, grouping only tightly packed points and classifying many points as outliers.

As eps increases to values like 0.3 or 0.5, DBSCAN forms larger, more coherent clusters and reduces the number of noise points. At eps = 0.5 with min_samples = 5, we observe well-defined clusters with minimal noise, suggesting that this combination provides a good balance for identifying species groupings without excessive noise points.

The min_samples parameter affects the minimum density required to form clusters. Lower min_samples values allow more clusters to form, while higher values require more points in close proximity, increasing the likelihood of points being classified as noise. For example, with eps = 0.5 and min_samples = 7, fewer points are classified as core points, resulting in slightly more noise compared to min_samples = 5.

6.2 Analysis of DBSCAN and Comparison with K-Means

DBSCAN and K-means offer different clustering perspectives due to their distinct methodologies. K-means assigns each point to the nearest cluster center, forming compact, spherical clusters, which works well for clearly separable clusters, like Iris-setosa. However, for overlapping clusters, such as Iris-versicolor and Iris-virginica, K-means struggles to capture complex structures, resulting in some misclassification.

In contrast, DBSCAN adapts to data density, forming clusters of arbitrary shapes and identifying points in sparse regions as noise. This allows it to capture non-spherical clusters and better separate overlapping species in the Iris dataset. DBSCAN's ability to label noise points (those in sparse areas) is useful for identifying outliers, whereas K-means assigns every point to a cluster, which can misclassify distant points.

K-means requires the number of clusters (k) to be specified in advance, making it less flexible for exploratory clustering. DBSCAN, however, doesn't need a predefined cluster count, but it is sensitive to eps and min_samples, which require careful tuning to avoid too many noise points or merged clusters.

Overall, K-means works well for well-separated, spherical clusters, while DBSCAN is better suited for datasets with irregular cluster shapes and overlapping boundaries, as it adapts to density and identifies noise. In the Iris dataset, K-means separates Iris-setosa effectively but struggles with Iris-versicolor and Iris-virginica overlap. DBSCAN provides a more natural clustering, labeling noise points and capturing complex cluster shapes.

7. Conclusion

This study demonstrates the effectiveness of using PCA in combination with both K-means and DBSCAN to analyze the Iris dataset. PCA reduces the data from four to two dimensions, retaining most of the dataset's variance and enabling clearer visualization. K-means clustering on PCA-reduced data effectively identifies distinct, spherical clusters, making it suitable for separable groups like Iris-setosa. However, K-means is less effective for overlapping species, such as Iris-versicolor and Iris-virginica, due to its rigid cluster shape assumptions.

By contrast, combining PCA with DBSCAN allows for more adaptable clustering of complex shapes and densities. DBSCAN's density-based approach identifies irregular clusters and labels sparse points as noise, making it better suited for datasets with overlapping groups or varied structures. Overall, PCA with K-means is well-suited to linearly separable clusters, while PCA with DBSCAN provides greater flexibility for non-spherical clusters. The choice of methods should be guided by the dataset's structure and analysis objectives.